

TECHNICAL REPORT · VERSION 5.3.0

HOW OPENFPL PICKS A SQUAD

OpenFPL Scout AI forecasts Fantasy Premier League points for every player and builds a legal 15-player squad for the coming Gameweek. This report documents what the system is trained on, how it is validated, exactly how accurate it proved to be on a season it was not trained on, and the places where it is known to be weak.

Models trained **14 Aug 2026**

Training rows **86,755**

Held-out season **2025/26**

Source **github.com/elcaiseri/OpenFPL-Scout-AI**

Ensemble **Ridge · XGBoost · CatBoost · MLP**

Live data **Official FPL API**

Executive summary

OpenFPL was evaluated on a complete Premier League season that was withheld from training — 29,747 player-gameweeks that no model scoring them had been fitted on. Every number in this report comes from that evaluation or from the training artifacts that produced it.

HOLDOUT RMSE

1.92

Points per player-gameweek, against a 2.35-point standard deviation in actual scores

RANK CORRELATION

0.70

Mean Spearman ρ between forecast and outcome, across all 38 gameweeks

SQUAD POINTS / GW

70.6

Versus 56.0 for FPL's own expected-points ranking, on identical rules

GAMEWEEKS WON

31/38

Against a 5-gameweek form squad; 32 of 38 against FPL's own expected points

What the evaluation showed

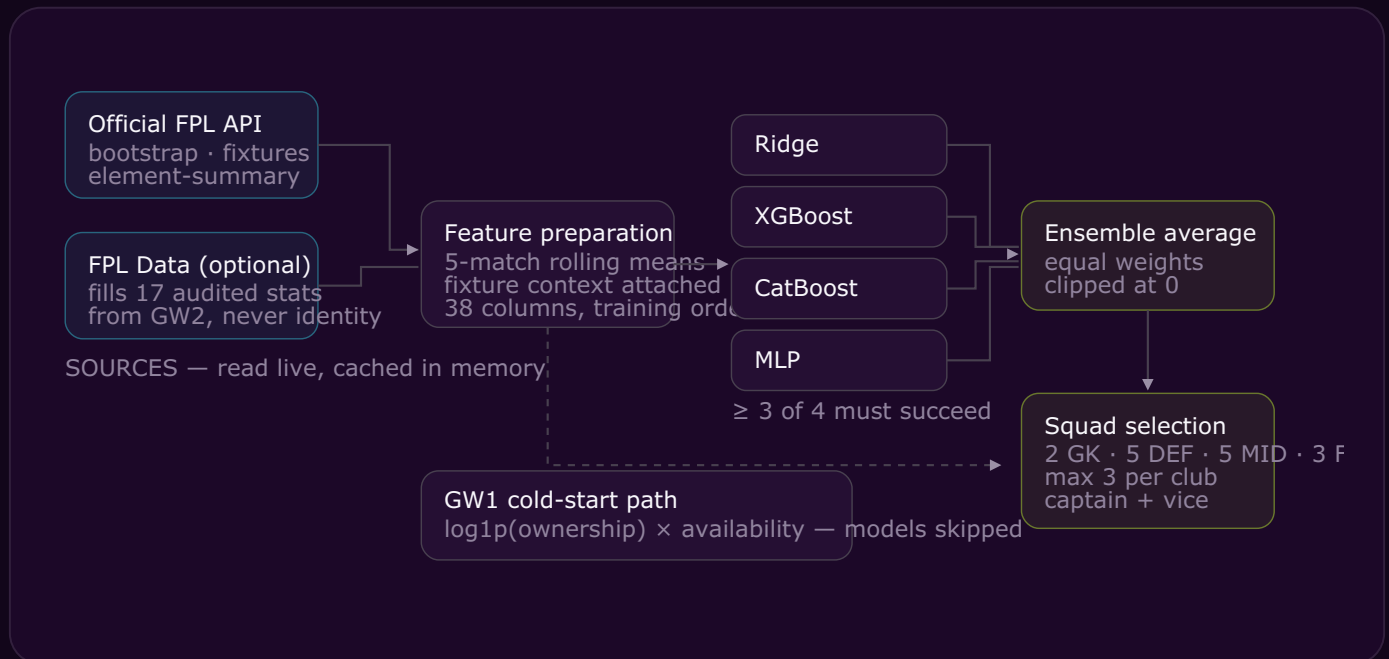
- **The ensemble beats every prediction baseline tested.** Against FPL's own published expected-points figure, a 5-gameweek rolling-form average, a last-gameweek-repeats rule, and a position-average rule, it had lower error on all three metrics — RMSE, MAE and R^2 .
- **Combining models helps, but only slightly.** The ensemble improves on the best single model, CatBoost, by under 0.1% RMSE — and CatBoost is fractionally *better* on MAE. The ensemble's real value is resilience: the service keeps working if a model fails to load or predict.
- **Ranking is the strength, not point estimates.** Individual scores carry about ± 1.9 points of error, which is irreducible in a game where a single goal is worth 4–6 points. Ordering players correctly is where the system earns its keep, and it does that consistently: $\rho \approx 0.72$ in every gameweek after the season opener.
- **Gameweek 1 is genuinely hard.** With no current-season match history, rank correlation collapses to 0.12. Production handles this with a separate ownership-based cold start rather than pretending the models have signal they do not have.

THE HONEST HEADLINE

OpenFPL will not tell you what a player is about to score. It will reliably tell you which players are worth more than others this week, and that ordering is good enough to build a squad that beat every naive strategy we measured it against over a full season.

System architecture

The service is a FastAPI application. Every request for a squad triggers a live read from the official FPL API, feature construction, four model predictions, and a constrained selection pass. Nothing is precomputed overnight.



Two paths exist. In the normal path, five-match rolling averages are built from official per-player history, fixture context for the target Gameweek is attached, and the four models each score every player. In Gameweek 1, when no current-season match history exists at all, the models are deliberately bypassed for an explicit ownership-and-availability heuristic — the app is currently serving this path, since the 2026/27 season has not yet kicked off.

Data and features

Training used 86,755 cleaned player-match rows across three Premier League seasons. Each row is one player in one fixture, and the target is the FPL points they actually scored.

Training corpus

- **2023/24** — 29,725 rows
- **2024/25** — 27,605 rows
- **2025/26** — 29,747 rows (held out)

After cleaning: **57,008** development rows and **29,747** holdout rows.

Feature construction

Every match statistic is replaced by that player's mean over their **previous five matches**, shifted so the match being predicted never contributes to its own inputs. Five categorical fields — position, player, club, opponent, and home/away — are kept raw.

38 model inputs in total.

What the model actually sees

The 32 rolling numerical inputs cover attacking volume (shots, shots on target, shots in box, touches in the box, carries into the final third and penalty area), expected-goal metrics (xG, npG, xA, xGI, npGI, xGC, xCS), realised output (goals, assists, clean sheets, goals conceded), defensive work (tackles, interceptions, clearances, blocks, recoveries, CBI, defensive contribution), and context (minutes, price, ownership, FPL's own xP, and points versus xP).

LEAKAGE CONTROL

Two specific traps are handled explicitly. First, rolling means are shifted by one match, so a player's own result never feeds their own prediction. Second, in a double gameweek both fixtures receive the history available at the *start* of that gameweek — fixture one's result is not leaked into fixture two, because in reality both forecasts are made before either match is played. Missing season-specific statistics are left as NaN and imputed inside each validation fold, so no information crosses from a validation season into training.

DATA PROVENANCE

Live inference is driven by the official Fantasy Premier League API, which is authoritative for player identity, availability, prices, fixtures, and match history. Historical model training additionally used public FPL Data statistics CSVs to supply 17 detailed inputs that official history does not expose. Permission for that source is recorded as *pending* in the project documentation; the integration is guarded, attributed, and can be disabled at runtime with a single environment variable (`FPL_DATA_INFERENCE_ENABLED=false`), after which the service runs on official data alone.

04

The model ensemble

Four deliberately different learners are trained on identical features and averaged. Diversity is the point: a linear model, two gradient-boosted tree families, and a neural network make different mistakes.

MODEL	ROLE	TUNED HYPERPARAMETERS
Ridge	Linear anchor; stable, high bias	<code>alpha = 25.0</code>
XGBoost	Gradient-boosted trees	<code>max_depth = 3, n_estimators = 300, lr = 0.03</code>
CatBoost	Ordered boosting, native categoricals	<code>depth = 5, iterations = 550, lr = 0.03</code>
MLP	Non-linear interactions	<code>hidden = (128, 64), alpha = 0.001, batch = 256</code>

Hyperparameters were selected by grid search scored under the same forward-chaining cross-validation described in the next section, never on the holdout. All four are shallow and heavily regularised by modern standards — depth 3 for XGBoost, depth 5 for CatBoost — which is what the signal in this problem supports.

How predictions are combined

At runtime the four predictions are combined as an **equal-weighted mean** and then clipped at zero, since a negative forecast is not meaningful. The trainer additionally fits optimised ensemble weights on out-of-fold predictions; on the holdout that weighted variant scores 1.9240 RMSE against 1.9245 for the equal-weighted mean the service actually uses — a difference of 0.03%, which is why the simpler and more predictable equal weighting is deployed.

The service requires at least **three of four** models to return finite predictions of the correct length. Below that it fails the request loudly rather than serving a degraded squad. Each model's stored feature contract is also verified against the current feature list at load time, so a stale artifact cannot silently produce nonsense.

05

Validation methodology

Football is a time series, so random shuffled splits would be dishonest — they let a model learn from May to predict September. Two independent time-respecting checks were used instead.



Five expanding folds

Each fold trains on everything up to a cut-off and validates on the period that follows, growing from 13,714 training rows in fold 1 to 71,183 in fold 5. This mirrors how the service is actually used: fit on the past, predict the next block of fixtures. Hyperparameters, model selection, and ensemble weights were all decided here.

One held-out season

The 2025/26 season was then set aside. All four models were refit on 2023/24 and 2024/25 only — 57,008 rows — and scored once against the 29,747 rows of 2025/26. Those predictions, which every accuracy figure in this report is computed from, come from estimators that had never been fitted on a single row of the season they were scored against.

HOW CLEAN THE HOLDOUT ACTUALLY IS

Two distinct things can leak: model *fitting* and hyperparameter *selection*. Fitting is clean here — the scored estimators saw only the two earlier seasons. Selection is not fully clean in the artifacts this report describes: the tuning grid was scored with the five folds above, and folds 4 and 5 validate inside 2025/26, the same season later used as the holdout. Hyperparameters were therefore chosen with some visibility of the holdout season.

The practical effect is small — the choice amounted to picking one of about five candidates per model, a far weaker channel than fitting, and the tuning curves are shallow — but it means the holdout figures should be read as **mildly optimistic rather than pristine**. The trainer currently in the repository restricts tuning and cross-validation to development rows only, which closes this gap; the shipped artifacts predate that change and a retrain would re-establish a fully untouched holdout.

WHAT THE HOLDOUT DOES AND DOES NOT CERTIFY

Holdout figures measure the *procedure*. The model files shipped in production are refit on all three seasons, which is standard practice and should perform slightly better than the numbers here — but those exact files have seen the holdout, so they cannot be scored on it. Read every accuracy figure in this report as **what this method achieved on a season it was not trained on**, not as a certificate for a specific binary.

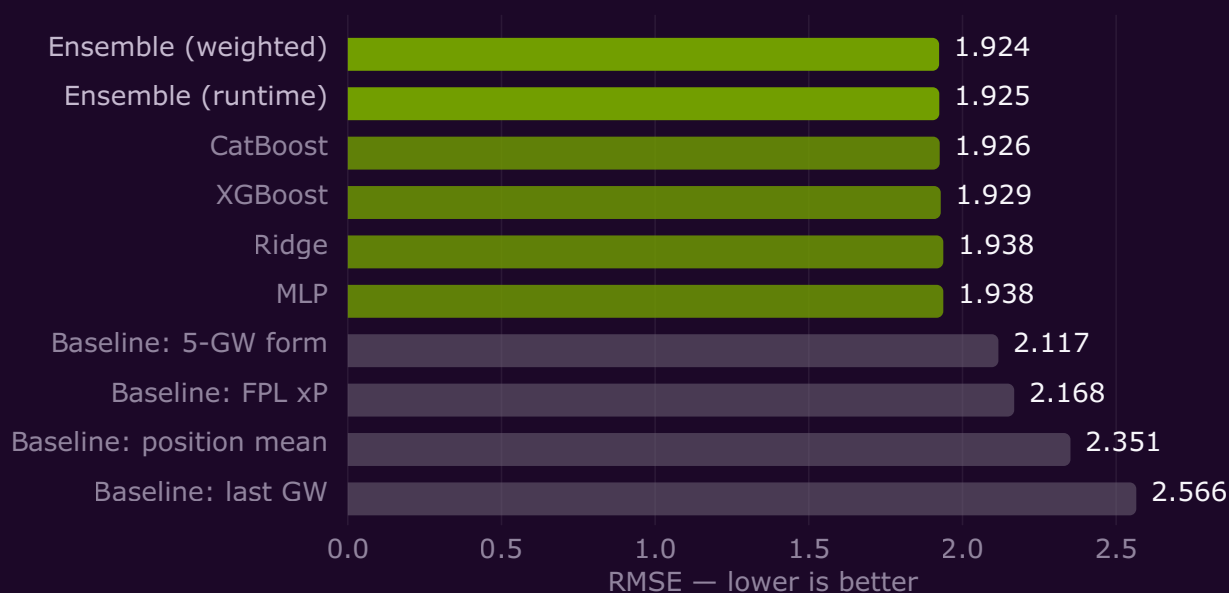
06

Forecast accuracy

Two views of the same question. First, how each model and baseline ranked on the held-out season. Second, how stable those models were across the five cross-validation folds.

Error on the held-out 2025/26 season

Root mean squared error across 29,747 player-gameweeks. Baselines shown in grey. Lower is better.



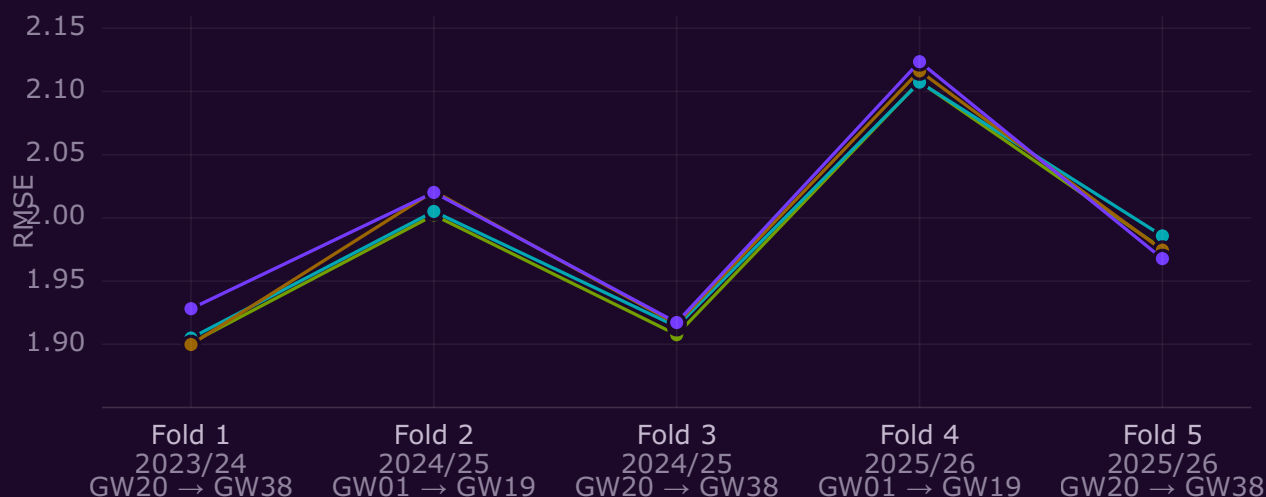
Reading this: the four models cluster tightly between 1.926 and 1.938, and averaging them buys only a further 0.001. The meaningful gap is to the baselines — the best of them, a 5-gameweek rolling form average, sits 0.19 RMSE worse, and FPL's own published expected-points figure is worse still. Actual points in this season had a standard deviation of 2.35, so an RMSE of 1.92 represents a real but bounded reduction in uncertainty.

MODEL	RMSE	MAE	R ²
Ensemble (weighted)	1.9240	0.9623	0.3307
Ensemble (runtime)	1.9245	0.9663	0.3303
CatBoost	1.9259	0.9617	0.3293
XGBoost	1.9293	0.9641	0.3270
Ridge	1.9376	0.9990	0.3212
MLP	1.9376	0.9944	0.3212
Baseline: 5-GW form	2.1169	1.0559	0.1897
Baseline: FPL xP	2.1683	1.1363	0.1499
Baseline: position mean	2.3514	1.4798	0.0003
Baseline: last GW	2.5656	1.1499	-0.1902

Stability across cross-validation folds

Validation RMSE per expanding fold. Each fold trains on all prior data and validates on the period that follows.

■ CatBoost ■ XGBoost ■ MLP ■ Ridge

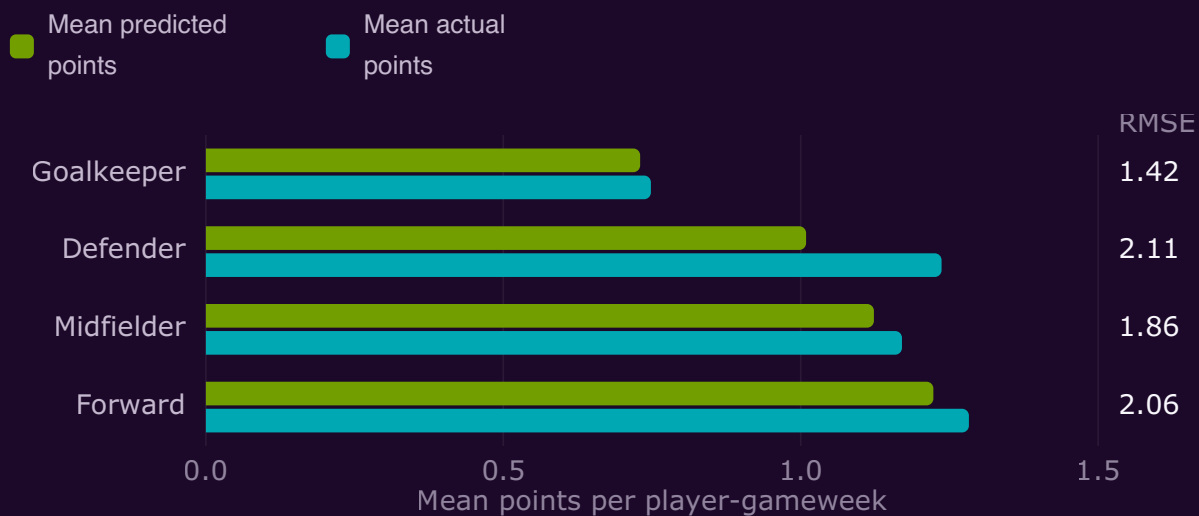


Reading this: all four models move together — fold 4 is the hardest for every one of them, and fold 1 the easiest for three of the four. That the curves are near-parallel is itself informative: the variation is driven by which fixtures fall in the validation window, not by model choice. It also explains why ensembling gains so little — models that fail in the same places cannot cover for each other. Fold 4 is the first fold asked to predict a season it has no data from, which is the single best predictor of how the system behaves at the start of a new campaign.

MODEL	CV RMSE	CV MAE	CV R ²	FIT + SCORE
CatBoost	1.9788 ± 0.084	1.0389	0.2738	17.2s
XGBoost	1.9834 ± 0.082	1.0436	0.2704	6.2s
MLP	1.9855 ± 0.088	1.0896	0.2690	31.2s
Ridge	1.9914 ± 0.084	1.0861	0.2648	2.1s

Accuracy by position

Mean predicted versus mean actual points, with RMSE, on the held-out season.



Reading this: goalkeepers are the most predictable group by a wide margin (RMSE 1.42) — they play whole matches and their scoring is dominated by clean sheets and saves. Defenders are the hardest (2.11), because a defender's score hinges on a binary clean sheet plus rare attacking returns. The model is also visibly conservative for defenders, predicting 1.01 against an actual mean of 1.24; it under-calls the attacking upside of defenders more than any other group.

POSITION	ROWS	RMSE	MAE	MEAN PREDICTED	MEAN ACTUAL
Goalkeeper	3,427	1.416	0.613	0.730	0.748
Defender	9,733	2.114	1.092	1.009	1.237
Midfielder	13,309	1.857	0.943	1.123	1.170
Forward	3,278	2.060	1.055	1.223	1.283

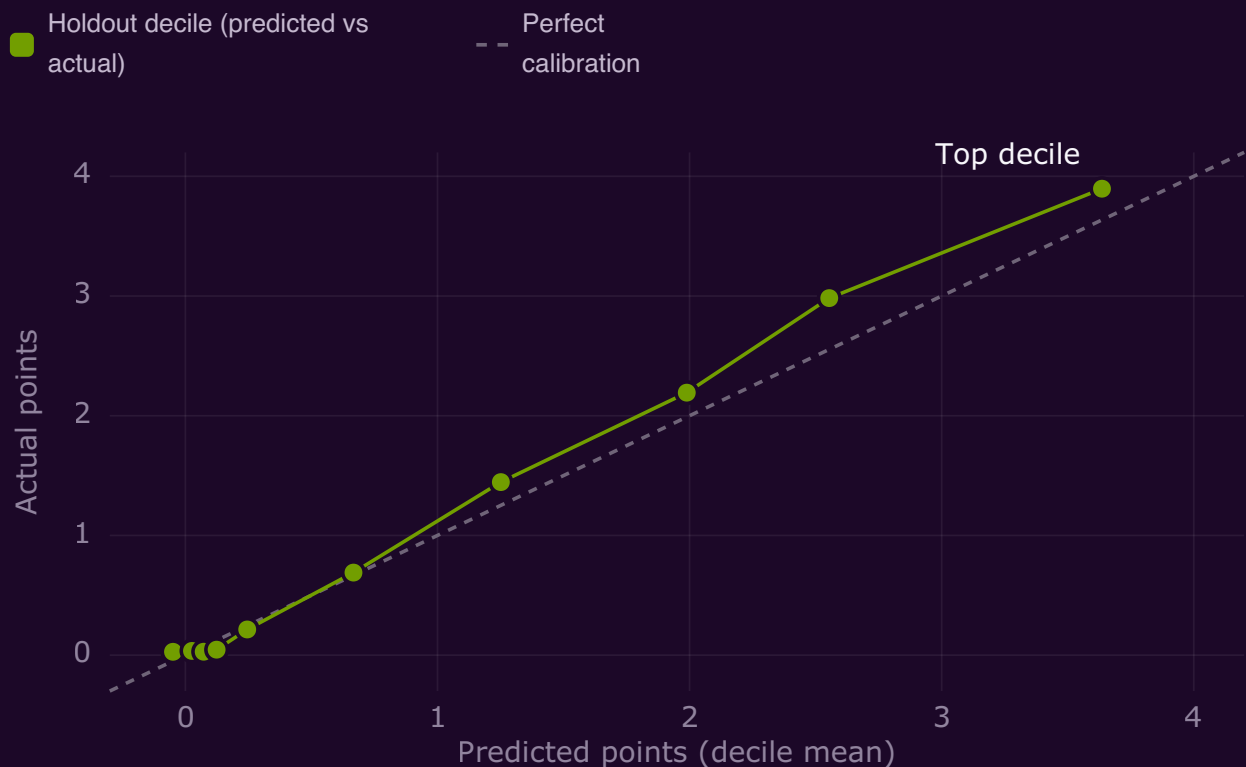
07

Calibration and ranking skill

Error magnitude is only half the story. For a squad picker, two other things matter more: do higher forecasts actually correspond to higher scores, and does that hold week to week?

Calibration by forecast decile

Holdout players grouped into ten equal buckets by forecast, then plotted against what they actually scored.



Reading this: the relationship is monotonic — every decile scored more than the one below it, which is precisely the property a ranking system needs. The curve tracks the diagonal closely in the middle of the range and rises *above* it at the top: the highest-forecast decile was predicted at 3.64 points and actually returned 3.90. The model systematically under-promises on its best picks, which is the safer direction to be wrong in. Deciles 3 and 4 sit just below the diagonal — mildly over-called — which is the fringe-player region where minutes are least certain.

Weekly ranking skill across the held-out season

Spearman rank correlation between forecast and actual points, computed separately for each gameweek.



Reading this: after the opener, ranking skill is remarkably flat — ρ between 0.65 and 0.76 in all 37 remaining gameweeks, averaging 0.72. There is no decay across the season and no single catastrophic week. Gameweek 1 is the visible exception at $\rho = 0.12$: with zero current-season history the models have nothing to work with. This is the measured justification for the separate cold-start path in production.

08

Squad selection backtest

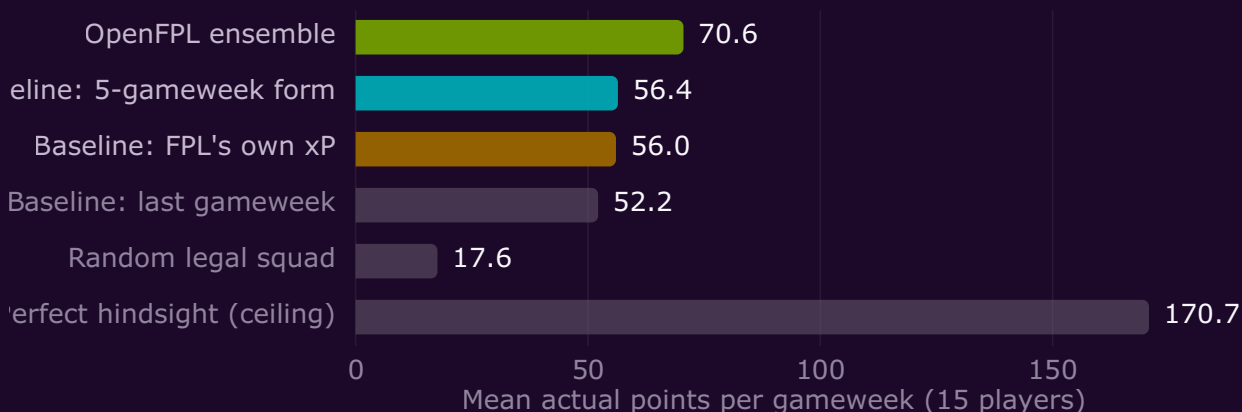
RMSE is an abstraction. The question users actually care about is whether the resulting squad scores more points. So the full selection was replayed for all 38 gameweeks of the held-out season, against the same strategies, under identical rules.

METHOD

For each gameweek, each strategy ranks every available player and fills the official positional quota — 2 goalkeepers, 5 defenders, 5 midfielders, 3 forwards — highest first. The score reported is the actual FPL points those 15 players went on to record. All strategies face the identical player pool and identical constraints; the only difference is the ranking they use.

Points per gameweek by selection strategy

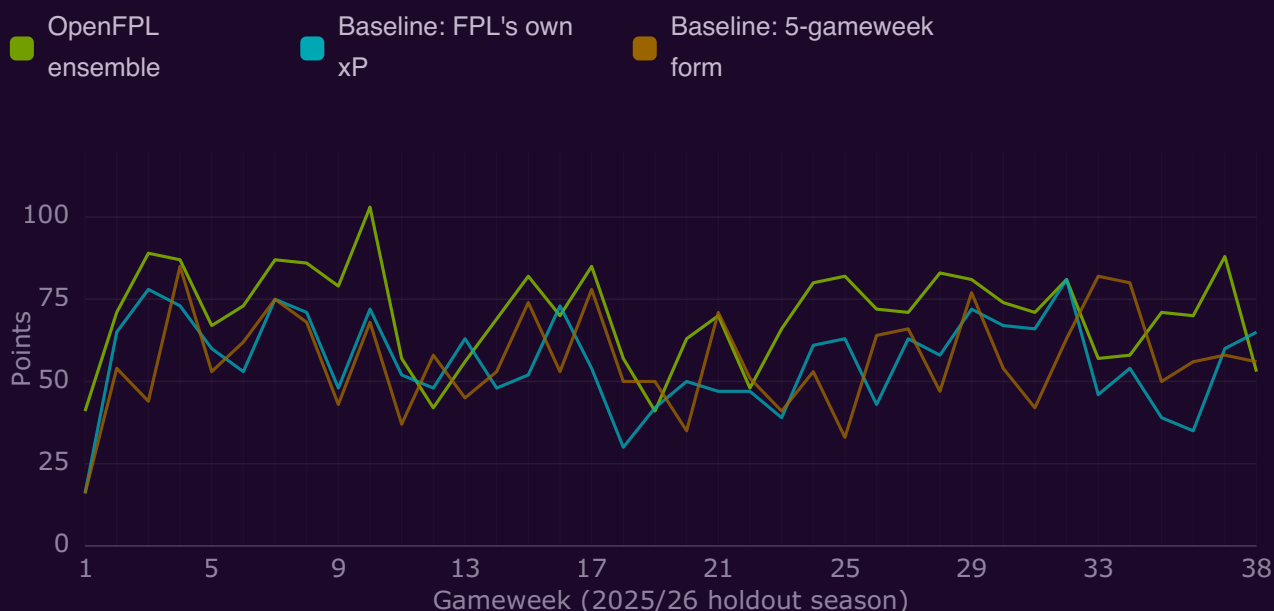
Mean actual points returned by the selected 15, averaged over the 38 gameweeks of the held-out season.



Reading this: the ensemble returned 70.6 points per gameweek against 56.4 for a 5-gameweek form ranking and 56.0 for FPL's own expected-points figure — roughly **+25%** over the best baseline, or about 540 extra points across a season. The two reference bars bound the problem: a random legal squad returns 17.6, and perfect hindsight returns 170.7. OpenFPL captures about 41% of the theoretical maximum.

Week by week, not just on average

Actual points returned by each strategy's squad in every gameweek of the held-out season.



Reading this: the advantage is persistent rather than driven by a handful of lucky weeks. The ensemble beat the 5-gameweek form squad in 31 of 38 gameweeks and FPL's expected-points squad in 32 of 38. It also loses sometimes, and visibly: gameweeks 12, 19 and 38 fell short of both baselines, and gameweek 33 was the single largest shortfall — 57 points against 82 for the form squad. Any individual gameweek is high variance; the edge shows up over a season, not in any one week.

STRATEGY	POINTS / GW	SEASON TOTAL	CAPTAIN PICK AVG	GWS BEATEN BY AI
OpenFPL ensemble	70.6	2,681	6.84	—
Baseline: 5-gameweek form	56.4	2,145	4.42	31 / 38
Baseline: FPL's own xP	56.0	2,129	4.53	32 / 38
Baseline: last gameweek	52.2	1,983	4.08	32 / 38
Random legal squad	17.6	669	—	38 / 38
Perfect hindsight (ceiling)	170.7	6,488	—	—

Captaincy

The captain is the single highest-forecast player in the squad, and doubling their score makes it the highest-leverage decision in the game. Across the held-out season, the ensemble's captain pick returned **6.84 points on average**, against 4.53 for FPL's expected-points ranking and 4.42 for a form ranking. Because the captain's score is counted a second time, that gap is worth roughly **2.3 extra points per gameweek** on top of the squad advantage shown above.

THREE HONEST CAVEATS ON THIS BACKTEST

It is not a simulated FPL season. It sums all 15 selected players rather than modelling a starting XI, bench order, or automatic substitutions, and it applies no budget, no transfer limits, and no chips. It measures ranking quality under positional constraints — nothing more.

The per-club limit is not applied. The production selector caps three players per club; the archived holdout predictions do not carry a club column, so the backtest enforces position quotas only. In practice this constraint costs points rather than adding them, so the live selector's true figure will be somewhat lower than 70.6.

Perfect hindsight is not a reachable target. The 170.7 ceiling assumes you already know every result. It is included to give the 70.6 a scale, not as a goal.

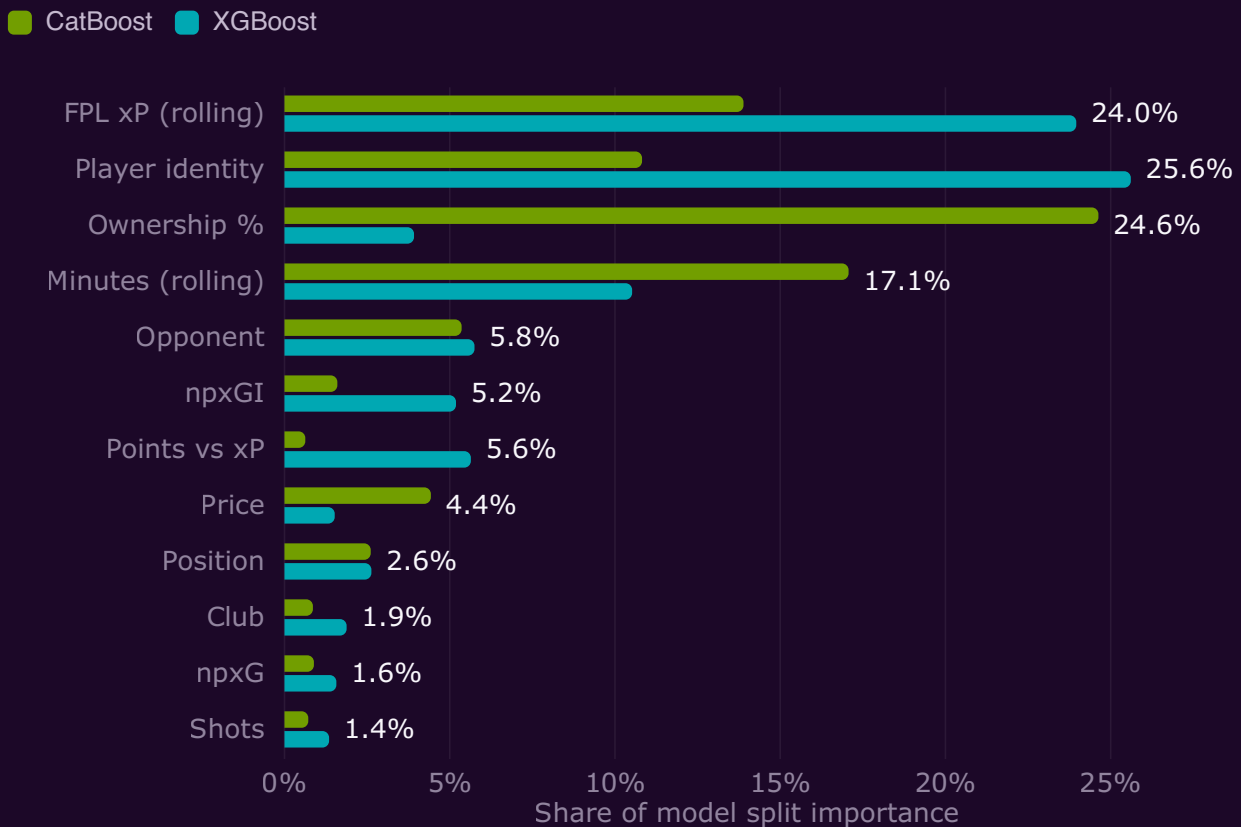
09

What drives a forecast

Both tree models expose which inputs they split on most. The two disagree in interesting ways, which is exactly why they are ensembled.

Top model inputs by split importance

Share of total importance, with one-hot encoded categorical columns summed back to their source field.



Reading this: four inputs dominate both models — rolling minutes, FPL's own xP, ownership, and player identity. That minutes rank so highly is the model rediscovering the first rule of FPL: a player who does not start cannot score. Note the disagreement. CatBoost leans heavily on ownership (24.6%), effectively using the crowd as a prior; XGBoost leans on player identity and xP instead. Importance shows what a model splits on, not what causes points — ownership predicts well because good players are widely owned, not the other way round.

10

Squad and captaincy rules

Selection is a greedy constrained pass over the forecast ranking, not an optimiser. Given the ranking, the outcome is deterministic and inspectable.

1. Drop any player without a valid position, club, or finite forecast.
2. Sort every remaining player by forecast, highest first.
3. Walk the list, taking each player unless their position quota (2 / 5 / 5 / 3) is full or their club already has three selected.
4. Stop at 15. Assign the highest forecast as captain and the second as vice-captain.

SELECTION IS BUDGET-FREE BY DESIGN

Prices are returned for context but play no part in either the forecast or the selection. The squad answers "who are the 15 best-projected players you can legally field together", not "what is the best squad for £100m". Treat it as a shortlist and a captaincy signal rather than a team sheet you can copy directly.

Gameweek 1

When no current-season match history exists, the models are skipped entirely. Players are ranked by $\log_{1p}(\text{ownership \%}) \times \text{availability}$, where availability is derived from official status flags and any published chance-of-playing percentage, and unavailable players are excluded outright. The response labels this path as `ownership-cold-start` so clients can tell the difference. The holdout data justifies the switch: GW1 rank correlation from the models alone was 0.12 against 0.72 for every other week.

Team rating explained

Any public FPL team ID can be scored from 0 to 100 against the same forecasts. The score is fully deterministic and decomposes into three published components.

Starting XI — 80 points

Your eleven starters' combined forecast, divided by the best legal XI the AI can build from its own benchmark squad, capped at 1.0 and scaled to 80.

Captaincy — 10 points

Your captain's forecast divided by the AI benchmark captain's forecast, capped at 1.0 and scaled to 10.

Availability — 10 points

The share of your eleven starters currently flagged as available, scaled to 10. Injured or suspended starters score zero here.

The three components sum and round to the published rating, which maps to a letter grade (A+ at 95 and above, down to E below 50). The response also returns your projected points, the AI benchmark's projected points, the gap between them, your differential count below 10% ownership, and plain-language strengths and risks.

TWO THINGS THE RATING IS NOT

It is **not budget-aware** — the benchmark it compares you to ignores price, so a well-built squad on a real budget will always trail it somewhat. And it rates the picks **officially published by FPL**, which for a future gameweek means your current lineup, not whatever you intend to do before the deadline.

Service engineering

API surface

33 endpoints across nine tags. Seven are public — the routes the web app itself uses — and 26 require a bearer token. The complete live catalogue is served at `/api`, with generated documentation at `/redoc`.

AREA	ENDPOINTS	PUBLIC	PURPOSE
Scout AI	5	2	Projections, squads, team ratings
Gameweeks	6	2	Event state, live scoring, dream teams
Reference & rankings	6	0	Regions, set pieces, winners
Managers	4	0	Profiles, history, transfers, picks
Leagues & cups	4	0	Standings and cup status
Players	3	1	Search, prices, availability, history
Fixtures	2	1	Fixtures, difficulty, per-fixture stats
Service	2	1	Catalogue and health
Teams	1	0	Clubs and strength ratings

Caching and upstream behaviour

Official responses are cached in memory for 300 seconds, per-player history for 900 seconds, and player history is fetched across up to eight worker threads. Optional FPL Data enrichment is cached for six hours, only applies from Gameweek 2, requires the exact configured season, demands at least an 80% player match rate, and never overwrites an official value. If any of that fails, the request completes on official data alone — enrichment can degrade the forecast but can never take the service down.

Measured latency

Measured locally against the running service with upstream caches warm. These are indicative only — they exclude network distance to the client, Cloud Run cold starts, and the several hundred milliseconds an uncached upstream fetch adds to the first request.

ENDPOINT	WARM RESPONSE
/api	1.3 ms
/api/fpl/gameweeks	1.7 ms
/api/fpl/fixtures?gameweek=1	2.0 ms
/api/fpl/players	10.9 ms
/api/scout?gameweek=1	17.5 ms

The scout figure above is the Gameweek 1 cold-start path, which does not invoke the models. The same endpoint's first uncached call, including the upstream fetch, measured 511 ms. A full four-model inference over the entire player pool is heavier than either figure.

Deployment

The container excludes generated data and model artifacts; both are mounted read-only at runtime. It defaults to port 8000 and honours Cloud Run's `PORT`. The documented low-traffic profile is 1 vCPU, 512 MiB, concurrency 4, scale-to-zero, and a three-instance cap.

Limitations

Everything below is a measured or structural property of the system, not a hypothetical.

Individual scores carry real error

RMSE 1.92 against an actual standard deviation of 2.35 means roughly 33% of the variance in FPL points is explained and two-thirds is not. A single goal is worth 4–6 points and arrives largely at random. Use the ordering; do not read a 5.2 forecast as a prediction of five points.

Gameweek 1 is a heuristic

With no current-season history the models are bypassed for an ownership-and-availability ranking. That is a crowd-following prior, not a forecast, and it inherits whatever the crowd gets wrong. The app is serving this path right now.

No budget, transfers, or chips

Selection ignores price entirely and models no transfer cost, bench order, automatic substitution, or chip strategy. The squad is a shortlist, not a directly playable team.

Defenders are the weak spot

Highest RMSE of any position at 2.11, and systematically under-forecast — a mean prediction of 1.01 against 1.24 actual. Clean sheets are binary events the model reads conservatively.

New players have thin history

Features are five-match rolling means. A summer signing, a promoted-club player, or anyone returning from long injury has little or no usable history, and player identity is itself a significant model input. Early-season forecasts for unfamiliar names are the least trustworthy the system produces.

Rotation and news are not modelled

Nothing ingests press conferences, manager comments, or rotation risk beyond official availability flags and rolling minutes. A rested starter looks identical to a guaranteed one until FPL flags them.

DISTRIBUTION SHIFT

The training corpus ends with 2025/26. The deployed models are therefore forecasting **2026/27, a season no model in this report has any data from** — a step beyond even the holdout, which at least sat adjacent to its training seasons. FPL also periodically changes its scoring rules, and squads turn over every summer.

Cross-validation fold 4 is the measured shape of this risk: it was the first fold asked to predict a season it had no data from, and it was the worst fold for all four models — RMSE about 0.2 higher than each model's easiest fold. Expect accuracy to be at its weakest early in a new season and after any scoring change, and to improve as the retraining corpus catches up.

Reproducibility

Everything described in this report is open source, including the figures on this page. If a number here looks wrong, you can check it yourself — and then tell us.

SOURCE AND CONTRIBUTIONS

The models, the training pipeline, the backtests behind every figure, the API, and this report all live in one repository:

github.com/elcaiseri/OpenFPL-Scout-AI

Fork it to run your own experiments, retrain on your own data, or try a different model. Open an issue if a figure looks wrong or a caveat is understated, and send a pull request if you improve something — contributions of any size are welcome.

```
git clone https://github.com/elcaiseri/OpenFPL-Scout-AI.git
cd OpenFPL-Scout-AI
```

Training is seeded (`random_seed = 42`) and every run writes its artifacts to `models/`: fitted pipelines, per-fold results, tuning candidates, out-of-fold predictions, holdout predictions, and a metadata file recording the dataset, feature list, validation strategy, and chosen hyperparameters. Every figure in this report is generated from those files.

```
uv sync --all-groups
uv run uvicorn main:app --reload      # serve at localhost:8000
uv run python -m scripts.collect_official_fpl --gameweek 39
```

ARTIFACTS PREDATE THE CURRENT TRAINER

The shipped artifacts come from a run on 14 August 2026 that used an earlier version of `trainer-booster.py`. Two signatures identify it: the cross-validation folds in `cv_fold_results.csv` span all 86,755 rows rather than the 57,008 development rows the current code passes to them, and `training_metadata.json` lacks the `holdout_season`, `development_rows`, and `holdout_rows` keys the current code writes. No `ensemble_weights` file was written either. This is what makes the selection caveat in section 05 apply. Retraining with the current trainer would regenerate every figure here on a strictly clean holdout.

Test suite status

48 tests pass across six modules covering the official FPL client, scout inference and selection, team rating, data enrichment, the download importer, and the API schema. Two caveats apply to running them:

- `tests/test_features.py` currently **fails to collect**: it imports `estimate_fixture_difficulty` from `src.features`, which no longer exists there. Those nine tests are not running and the module needs updating to match the current feature API.
- `pytest` is not declared in `pyproject.toml`, and collection requires `PYTHONPATH=.`. A plain `uv run pytest` picks up an interpreter without the project's runtime dependencies and fails on import.

```
PYTHONPATH=. uv run --with pytest pytest -q
```

Artifact provenance

ARTIFACT	CONTENTS
<code>training_metadata.json</code>	Dataset summary, feature list, fold results, tuning, seed, timestamp
<code>holdout_predictions.csv</code>	29,747 rows — actuals, four model outputs, four baselines, both ensembles
<code>oof_predictions.csv</code>	Out-of-fold predictions used to fit ensemble weights
<code>cv_fold_results.csv</code>	Per-model, per-fold RMSE, MAE, R ² , and fold boundaries
<code>tuning_results.csv</code>	Every hyperparameter candidate with its cross-validated score
<code>*_reg.pkl</code>	Fitted scikit-learn pipelines, feature contract embedded

OpenFPL Scout AI · Technical report for version 5.3.0 · Models trained 14 August 2026 · Report generated 16 August 2026.

Live data from the official Fantasy Premier League API, with optional FPL Data enrichment. OpenFPL is an independent project and is not affiliated with, endorsed by, or connected to the Premier League or Fantasy Premier League. Forecasts are statistical estimates and carry the error documented above — they are not advice or a guarantee of any outcome.

Open source at github.com/elcaiseri/OpenFPL-Scout-AI — forks, issues, and pull requests welcome.

[Back to the app](#) · [API catalogue](#) · [API reference](#) · [@elcaiseri](#), 2026